МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №8

з дисципліни **"Функціональне програмування"**

на тему **"АЛГЕБРАЇЧНІ ТИПИ ДАНИХ "ПО-PYTHON'ІВСЬКИ""**

Варіант 13. Платіжні операції

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: змоделювати алгебраїчний тип даних засобами Python — незмінними `dataclass`-вузлами, `Union`/`TypeAlias`, `Literal` та `Protocol`; реалізувати чисті операції над ним через структурний pattern matching `match/case` і перевірити типи `mypy --strict`.

Теоретичні відомості:

Алгебраїчний тип даних — «сума варіантів»: значення належить рівно одному з кількох випадків, кожен зі своїми полями. У Python кожен випадок описують як `@dataclass(frozen=True)`, а сам тип — як `Union` (псевдонім через `TypeAlias`). `Literal` звужує поле до фіксованого набору значень (валюта, платіжна мережа), тож mypy відхилить невідому валюту ще до запуску. `Protocol` задає структурний контракт: будь-який об'єкт із потрібними атрибутами підходить без наслідування.

Структурний pattern matching `match/case` розбирає значення за формою: `case CardPayment(amount=a, network="visa")` одночасно перевіряє тип, прив'язує поля та звіряє літерал. Це замінює ланцюжки `isinstance`/`if-elif`, а для рекурсивних структур (пакет платежів, що містить інші пакети) дає природний рекурсивний обхід. Оскільки вузли незмінні, усі функції над ADT чисті: вони повертають нові значення, а не змінюють вхід.

Хід роботи:

1. У `model.py` описав ADT платежів за варіантом 13: `Literal`-псевдоніми `Currency` та `CardNetwork`, `Protocol PaymentSource` (сума й валюта), незмінні

вузли `CardPayment`, `BankTransfer`, `CashPayment` і рекурсивний `Batch` з кортежем вкладених платежів; тип `Payment` — їх `Union`:

```python
Currency: TypeAlias = Literal["UAH", "USD", "EUR"]
CardNetwork: TypeAlias = Literal["visa", "mastercard"]


class PaymentSource(Protocol):
    """Структурний контракт: будь-що з сумою та валютою."""

    @property
    def amount(self) -> float: ...

    @property
    def currency(self) -> Currency: ...


@dataclass(frozen=True)
class CardPayment:
    amount: float
    currency: Currency
    network: CardNetwork
    last4: str


@dataclass(frozen=True)
class BankTransfer:
    amount: float
    currency: Currency
    iban: str
    urgent: bool = False


@dataclass(frozen=True)
class CashPayment:
    amount: float
    currency: Currency

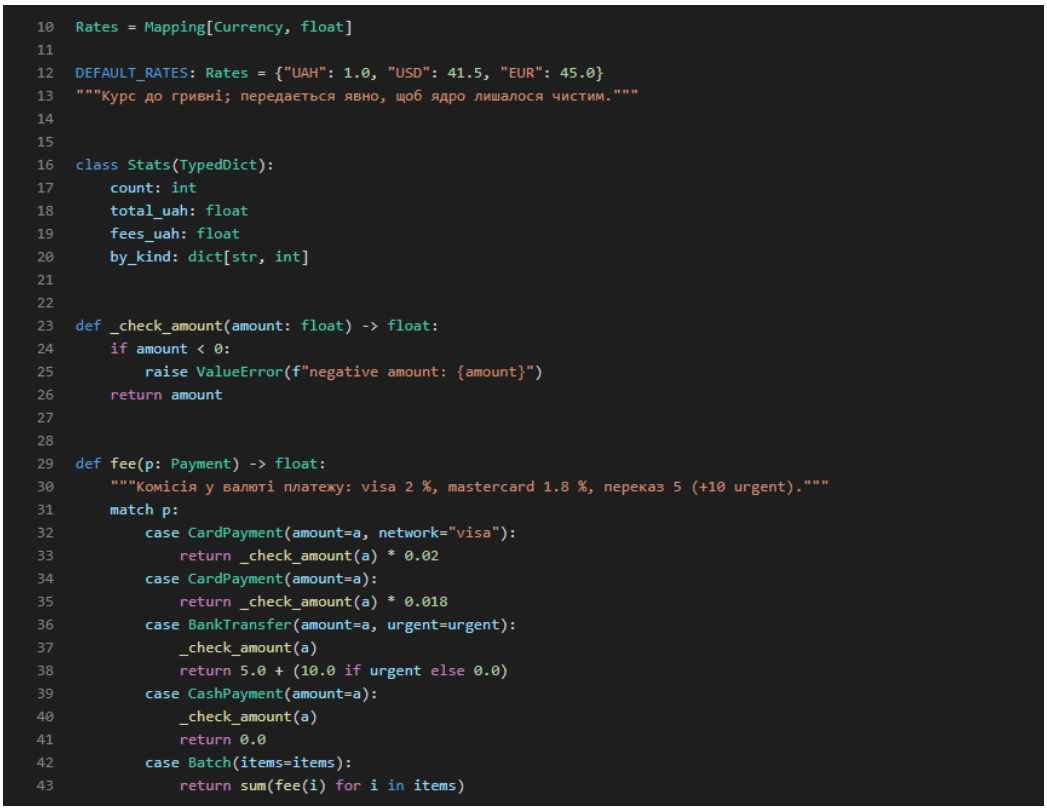

@dataclass(frozen=True)
class Batch:
    """Вкладена структура: пакет платежів, що може містити інші пакети."""

    items: tuple[Payment, ...]
    label: str = ""


Payment: TypeAlias = CardPayment | BankTransfer | CashPayment | Batch
```

2. В `evaluator.py` реалізував чисту функцію `fee` через `match/case`: для картки комісія залежить від мережі (літерал `"visa"` у шаблоні), переказ має фіксовану

плату з надбавкою за терміновість, готівка без комісії, пакет — рекурсивна сума; від'ємна сума дає `ValueError`:



```python
Rates = Mapping[Currency, float]

DEFAULT_RATES: Rates = {"UAH": 1.0, "USD": 41.5, "EUR": 45.0}
"""Курс до гривні; передається явно, щоб ядро лишалося чистим."""


class Stats(TypedDict):
    count: int
    total_uah: float
    fees_uah: float
    by_kind: dict[str, int]


def _check_amount(amount: float) -> float:
    if amount < 0:
        raise ValueError(f"negative amount: {amount}")
    return amount


def fee(p: Payment) -> float:
    """Комісія у валюті платежу: visa 2 %, mastercard 1.8 %, переказ 5 (+10 urgent)."""
    match p:
        case CardPayment(amount=a, network="visa"):
            return _check_amount(a) * 0.02
        case CardPayment(amount=a):
            return _check_amount(a) * 0.018
        case BankTransfer(amount=a, urgent=urgent):
            _check_amount(a)
            return 5.0 + (10.0 if urgent else 0.0)
        case CashPayment(amount=a):
            _check_amount(a)
            return 0.0
        case Batch(items=items):
            return sum(fee(i) for i in items)
```

3. Додав `convert` з явною таблицею курсів (параметр, а не глобальний стан; невідома валюта → `KeyError`), `total_with_fee` зі зведенням до заданої валюти

та рекурсією для `Batch`, а також `kind` і `flatten` для розгортання вкладених пакетів у кортеж листових платежів:

```python
def convert(
    amount: float, src: Currency, dst: Currency, rates: Rates = DEFAULT_RATES
) -> float:
    """Конвертація через гривню за явною таблицею курсів."""
    if src not in rates or dst not in rates:
        raise KeyError(f"unknown currency: {src if src not in rates else dst}")
    return amount * rates[src] / rates[dst]


def total_with_fee(
    p: Payment, currency: Currency = "UAH", rates: Rates = DEFAULT_RATES
) -> float:
    """Сума плюс комісія, зведена до заданої валюти (рекурсивно для Batch)."""
    match p:
        case Batch(items=items):
            return sum(total_with_fee(i, currency, rates) for i in items)
        case (
            CardPayment(amount=a, currency=c)
            | BankTransfer(amount=a, currency=c)
            | CashPayment(amount=a, currency=c)
        ):
            return convert(_check_amount(a) + fee(p), c, currency, rates)


def kind(p: Payment) -> str:
    """Назва виду платежу для статистики."""
    match p:
        case CardPayment():
            return "card"
        case BankTransfer():
            return "transfer"
        case CashPayment():
            return "cash"
        case Batch():
            return "batch"


def flatten(p: Payment) -> tuple[Payment, ...]:
    """Усі листові платежі пакета у порядку обходу (новий кортеж)."""
    match p:
        case Batch(items=items):
            return tuple(leaf for i in items for leaf in flatten(i))
        case _:
            return (p,)

```

4. Функція `statistics` агрегує кількість платежів, загальну суму й комісії у гривні та розподіл за видами, не змінюючи вхідні дані; `currency_of` дістає

валюту через or-шаблон:

```python
def statistics(payments: tuple[Payment, ...], rates: Rates = DEFAULT_RATES) -> Stats:
    """Агрегат: кількість листових платежів, суми у гривні, розподіл за видами."""
    leaves = tuple(leaf for p in payments for leaf in flatten(p))
    by_kind: dict[str, int] = {}
    for leaf in leaves:
        by_kind = {**by_kind, kind(leaf): by_kind.get(kind(leaf), 0) + 1}
    return {
        "count": len(leaves),
        "total_uah": sum(total_with_fee(leaf, "UAH", rates) for leaf in leaves),
        "fees_uah": sum(
            convert(fee(leaf), currency_of(leaf), "UAH", rates) for leaf in leaves
        ),
        "by_kind": by_kind,
    }


def currency_of(p: Payment) -> Currency:
    """Валюта листового платежу; для Batch — UAH (сума вже зведена)."""
    match p:
        case (
            CardPayment(currency=c) | BankTransfer(currency=c) | CashPayment(currency=c)
        ):
            return c
        case Batch():
            return "UAH"
```

5. У `pretty.py` описав `Protocol` `Renders` і рекурсивний `describe`, який рендерить платіж у текст; пакети виводяться з відступом за рівнем вкладеності, терміновий переказ має власний шаблон з літералом `urgent=True`:

```python
class Renders(Protocol):
    """Структурний контракт для будь-чого, що вміє відрендерити себе."""

    def render(self) -> str: ...


def describe(p: Payment, indent: int = 0) -> str:
    """Багаторядковий опис платежу; пакети рендеряться рекурсивно з відступом."""
    pad = "  " * indent
    match p:
        case CardPayment(amount=a, currency=c, network=n, last4=l4):
            return f"{pad}card {n} ****{l4}: {a:.2f} {c} (fee {fee(p):.2f})"
        case BankTransfer(amount=a, currency=c, iban=iban, urgent=True):
            return f"{pad}urgent transfer to {iban}: {a:.2f} {c} (fee {fee(p):.2f})"
        case BankTransfer(amount=a, currency=c, iban=iban):
            return f"{pad}transfer to {iban}: {a:.2f} {c} (fee {fee(p):.2f})"
        case CashPayment(amount=a, currency=c):
            return f"{pad}cash: {a:.2f} {c}"
        case Batch(items=items, label=label):
            head = f"{pad}batch {label or '(unnamed)'} [{len(items)}]"
            return "\n".join([head, *(describe(i, indent + 1) for i in items)])
```

6. Базове завдання з методички: AST `Expr = Num | Add | Mul` і альтернативна

форма з одним бінарним вузлом `Bin`, де оператор — `Literal["+", "*"]`:

```python
@dataclass(frozen=True)
class Num:
    value: int


@dataclass(frozen=True)
class Add:
    left: Expr
    right: Expr


@dataclass(frozen=True)
class Mul:
    left: Expr
    right: Expr


Expr: TypeAlias = Num | Add | Mul

Op: TypeAlias = Literal["+", "*"]


@dataclass(frozen=True)
class Num2:
    value: int


@dataclass(frozen=True)
class Bin:
    op: Op
    left: Expr2
    right: Expr2


Expr2: TypeAlias = Num2 | Bin
```

```python
class SupportsBin(Protocol):
    """Структурний контракт бінарного вузла."""

    @property
    def op(self) -> Op: ...

    @property
    def left(self) -> Expr2: ...

    @property
    def right(self) -> Expr2: ...


def eval_expr2(e: Expr2) -> int:
    """Чистий обчислювач альтернативного AST з оператором-літералом."""
    match e:
        case Num2(value=v):
            return v
        case Bin(op="+", left=a, right=b):
            return eval_expr2(a) + eval_expr2(b)
        case Bin(op="*", left=a, right=b):
            return eval_expr2(a) * eval_expr2(b)
        case _:
            raise ValueError(f"unknown node: {e!r}")
```

7. Чистий обчислювач `eval_expr` у `eval_.py` з перевіркою діапазону

(`EvalOverflow`) та `render` для дужкового запису:

```python
MAX_ABS: Final[int] = 10_000


class EvalOverflow(ValueError):
    """Результат вийшов за межі ±MAX_ABS."""


def _check_range(x: int) -> int:
    if abs(x) > MAX_ABS:
        raise EvalOverflow(f"overflow: {x}")
    return x


def eval_expr(e: Expr) -> int:
    """Обчислює вираз рекурсивно; кожен проміжний результат перевіряється."""
    match e:
        case Num(value=v):
            return _check_range(v)
        case Add(left=a, right=b):
            return _check_range(eval_expr(a) + eval_expr(b))
        case Mul(left=a, right=b):
            return _check_range(eval_expr(a) * eval_expr(b))


def render(e: Expr) -> str:
    """Повністю дужковий текст виразу."""
    match e:
        case Num(value=v):
            return str(v)
        case Add(left=a, right=b):
            return f"({render(a)} + {render(b)})"
        case Mul(left=a, right=b):
            return f"({render(a)} * {render(b)})"
```

8. Демонстраційний `main.py` будує вкладений пакет (картка, терміновий переказ у доларах, пакет готівкових платежів), друкує опис, суми у гривні та доларах, статистику й обчислення виразів:

```python
def main() -> None:
    """Друкує опис пакета платежів, суми та статистику."""
    batch = Batch(
        label="order-42",
        items=(
            CardPayment(1000.0, "UAH", "visa", "4242"),
            BankTransfer(100.0, "USD", "UA123456", urgent=True),
            Batch(
                label="tips", items=(CashPayment(50.0, "UAH"), CashPayment(5.0, "EUR"))
            ),
        ),
    )
    print(describe(batch))
    print(f"total with fees: {total_with_fee(batch):.2f} UAH")
    print(f"total with fees: {total_with_fee(batch, 'USD'):.2f} USD")
    print("stats:", statistics((batch,)))

    ast = Mul(Add(Num(2), Num(3)), Num(4))
    print("(2 + 3) * 4 =", eval_expr(ast))
    print("Bin:", eval_expr2(Bin("+", Num2(2), Bin("*", Num2(3), Num2(4)))))


if __name__ == "__main__":
    main()
```

9. Написав pytest-тести для платежів: комісії кожного виду, рекурсивна сума для пакета, конвертація з власною таблицею курсів, від'ємна сума та невідома

валюта, незмінність і хешованість вузлів:

```python
CARD = CardPayment(1000.0, "UAH", "visa", "4242")
MC = CardPayment(1000.0, "UAH", "mastercard", "1111")
TRANSFER = BankTransfer(100.0, "USD", "UA123456")
URGENT = BankTransfer(100.0, "USD", "UA123456", urgent=True)
CASH = CashPayment(50.0, "UAH")
BATCH = Batch(
    label="order", items=(CARD, URGENT, Batch(items=(CASH, CashPayment(5.0, "EUR"))))
)


def test_fee_per_kind() -> None:
    assert fee(CARD) == pytest.approx(20.0)
    assert fee(MC) == pytest.approx(18.0)
    assert fee(TRANSFER) == 5.0
    assert fee(URGENT) == 15.0
    assert fee(CASH) == 0.0


def test_fee_batch_is_recursive_sum() -> None:
    assert fee(BATCH) == pytest.approx(20.0 + 15.0 + 0.0 + 0.0)
    assert fee(Batch(items=())) == 0.0


def test_total_with_fee_and_conversion() -> None:
    assert total_with_fee(CARD) == pytest.approx(1020.0)
    assert total_with_fee(URGENT) == pytest.approx(115.0 * 41.5)
    assert total_with_fee(CASH, "USD") == pytest.approx(50.0 / 41.5)
    expected = 1020.0 + 115.0 * 41.5 + 50.0 + 5.0 * 45.0
    assert total_with_fee(BATCH) == pytest.approx(expected)
    assert total_with_fee(BATCH, "EUR") == pytest.approx(expected / 45.0)


def test_convert_with_custom_rates() -> None:
    rates: dict[Currency, float] = {"UAH": 1.0, "USD": 40.0, "EUR": 40.0}
    assert convert(10.0, "USD", "EUR", rates) == 10.0
    assert convert(1.0, "USD", "UAH", rates) == 40.0
    with pytest.raises(KeyError):
        convert(1.0, "USD", "GBP", rates)  # type: ignore[arg-type]


def test_negative_amount_rejected() -> None:
    with pytest.raises(ValueError):
        fee(CardPayment(-1.0, "UAH", "visa", "0000"))
    with pytest.raises(ValueError):
        total_with_fee(Batch(items=(CASH, CashPayment(-5.0, "UAH"))))


def test_nodes_are_immutable() -> None:
    with pytest.raises(FrozenInstanceError):
        CARD.amount = 1.0  # type: ignore[misc]
    assert CARD == CardPayment(1000.0, "UAH", "visa", "4242")
    assert len({CARD, CardPayment(1000.0, "UAH", "visa", "4242"), MC}) == 2

```

10. Тести `flatten`/`kind`, статистики (включно з порожнім входом), рендеру

вкладеного пакета та структурного контракту `PaymentSource`:

```python
def test_flatten_and_kind() -> None:
    leaves = flatten(BATCH)
    assert leaves == (CARD, URGENT, CASH, CashPayment(5.0, "EUR"))
    assert [kind(p) for p in leaves] == ["card", "transfer", "cash", "cash"]
    assert kind(BATCH) == "batch" and flatten(CASH) == (CASH,)


def test_statistics() -> None:
    s = statistics((BATCH, MC))
    assert s["count"] == 5
    assert s["by_kind"] == {"card": 2, "transfer": 1, "cash": 2}
    assert s["fees_uah"] == pytest.approx(20.0 + 15.0 * 41.5 + 18.0)
    assert s["total_uah"] == pytest.approx(total_with_fee(BATCH) + 1018.0)
    assert statistics(()) == {
        "count": 0,
        "total_uah": 0.0,
        "fees_uah": 0.0,
        "by_kind": {},
    }


def test_describe_nested() -> None:
    text = describe(BATCH)
    assert text.splitlines() == [
        "batch order [3]",
        "  card visa ****4242: 1000.00 UAH (fee 20.00)",
        "  urgent transfer to UA123456: 100.00 USD (fee 15.00)",
        "  batch (unnamed) [2]",
        "    cash: 50.00 UAH",
        "    cash: 5.00 EUR",
    ]
    assert describe(TRANSFER) == "transfer to UA123456: 100.00 USD (fee 5.00)"


def test_protocol_and_currency() -> None:
    def total_of(src: PaymentSource) -> str:
        return f"{src.amount} {src.currency}"

    assert total_of(CARD) == "1000.0 UAH" and total_of(CASH) == "50.0 UAH"
    assert currency_of(URGENT) == "USD" and currency_of(BATCH) == "UAH"
    assert DEFAULT_RATES["UAH"] == 1.0
    p: Payment = CASH
    assert isinstance(p, CashPayment)
```

11. Тести базового завдання з методички: обчислення, вкладений вираз, рендер, переповнення та форма `Bin`:

```python
def test_eval_basic() -> None:
    assert eval_expr(Num(5)) == 5
    assert eval_expr(Add(Num(2), Num(3))) == 5
    assert eval_expr(Mul(Num(2), Num(3))) == 6


def test_eval_nested() -> None:
    ast = Mul(Add(Num(2), Num(3)), Num(4))
    assert eval_expr(ast) == 20


def test_render() -> None:
    ast = Mul(Add(Num(2), Num(3)), Num(4))
    assert render(ast) == "((2 + 3) * 4)"


def test_overflow() -> None:
    with pytest.raises(EvalOverflow):
        eval_expr(Mul(Num(1000), Num(1000)))


def test_bin_literal_form() -> None:
    assert eval_expr2(Bin("+", Num2(2), Bin("*", Num2(3), Num2(4)))) == 14
```

12. Property-based тести на Hypothesis з рекурсивними стратегіями: для довільного вкладеного пакета сума і комісія дорівнюють сумі по листках,

статистика узгоджена з `flatten`, а рендер випадкового виразу обчислюється до того ж значення, що й `eval_expr`:

```python
amount = st.floats(min_value=0, max_value=1e5, allow_nan=False)
currency = st.sampled_from(["UAH", "USD", "EUR"])
leaf = st.one_of(
    st.builds(
        CardPayment,
        amount,
        currency,
        st.sampled_from(["visa", "mastercard"]),
        st.just("0000"),
    ),
    st.builds(BankTransfer, amount, currency, st.just("UA1"), st.booleans()),
    st.builds(CashPayment, amount, currency),
)
payment = st.recursive(
    leaf,
    lambda inner: st.builds(Batch, st.lists(inner, max_size=4).map(tuple)),
    max_leaves=12,
)


@given(payment)
def test_batch_total_equals_sum_of_leaves(p: Payment) -> None:
    leaves = flatten(p)
    assert abs(total_with_fee(p) - sum(total_with_fee(x) for x in leaves)) < 1e-6
    assert abs(fee(p) - sum(fee(x) for x in leaves)) < 1e-6


@given(st.lists(payment, max_size=5).map(tuple))
def test_statistics_count_matches_flatten(ps: tuple[Payment, ...]) -> None:
    s = statistics(ps)
    assert s["count"] == sum(len(flatten(p)) for p in ps) == sum(s["by_kind"].values())
    assert s["total_uah"] >= s["fees_uah"] >= 0.0


expr = st.recursive(
    st.builds(Num, st.integers(min_value=0, max_value=9)),
    lambda inner: st.one_of(st.builds(Add, inner, inner), st.builds(Mul, inner, inner)),
    max_leaves=6,
)


@given(expr)
def test_render_is_evaluable_python(e: Expr) -> None:
    try:
        value = eval_expr(e)
    except ValueError:
        return
    assert value == eval(render(e))  # noqa: S307 — рядок згенеровано нашим render
```

13. Запустив демонстрацію: пакет із трьох елементів рендериться з відступами, загальна сума з комісіями зведена до гривні та доларів:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab08> python main.py
batch order-42 [3]
  card visa ****4242: 1000.00 UAH (fee 20.00)
  urgent transfer to UA123456: 100.00 USD (fee 15.00)
  batch tips [2]
    cash: 50.00 UAH
    cash: 5.00 EUR
total with fees: 6067.50 UAH
total with fees: 146.20 USD
stats: {'count': 4, 'total_uah': 6067.5, 'fees_uah': 642.5, 'by_kind': {'card': 1, 'transfer': 1, 'cash': 2}}
(2 + 3) * 4 = 20
Bin: 14
```

14. Перевірив проєкт автоматичними інструментами: усі 18 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab08> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 18 items

tests\test_expr.py::test_eval_basic PASSED                               [  5%]
tests\test_expr.py::test_eval_nested PASSED                              [ 11%]
tests\test_expr.py::test_render PASSED                                   [ 16%]
tests\test_expr.py::test_overflow PASSED                                 [ 22%]
tests\test_expr.py::test_bin_literal_form PASSED                         [ 27%]
tests\test_payments.py::test_fee_per_kind PASSED                         [ 33%]
tests\test_payments.py::test_fee_batch_is_recursive_sum PASSED           [ 38%]
tests\test_payments.py::test_total_with_fee_and_conversion PASSED        [ 44%]
tests\test_payments.py::test_convert_with_custom_rates PASSED            [ 50%]
tests\test_payments.py::test_negative_amount_rejected PASSED             [ 55%]
tests\test_payments.py::test_nodes_are_immutable PASSED                  [ 61%]
tests\test_payments.py::test_flatten_and_kind PASSED                     [ 66%]
tests\test_payments.py::test_statistics PASSED                           [ 72%]
tests\test_payments.py::test_describe_nested PASSED                      [ 77%]
tests\test_payments.py::test_protocol_and_currency PASSED                [ 83%]
tests\test_properties.py::test_batch_total_equals_sum_of_leaves PASSED   [ 88%]
tests\test_properties.py::test_statistics_count_matches_flatten PASSED   [ 94%]
tests\test_properties.py::test_render_is_evaluable_python PASSED         [100%]

============================= 18 passed in 1.32s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab08> python -m mypy --strict .
Success: no issues found in 10 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab08> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab08> python -m black --check .
All done! ✨ 🍰 ✨
10 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я змоделював платіжні операції (варіант 13) як алгебраїчний тип даних: незмінні `dataclass`-вузли, `Union`-псевдонім `Payment`, `Literal` для валют і платіжних мереж та `Protocol` для структурного контракту. Усі операції — комісія, зведення до валюти, статистика, текстовий опис — реалізовані чистими функціями через `match/case` без ланцюжків `isinstance`, а рекурсивний вузол `Batch` показав, як pattern matching природно обходить вкладені структури. Юніт- і property-тести з рекурсивними стратегіями Hypothesis, `mypy --strict`, ruff і black підтвердили коректність і типобезпечність моделі.